TECHNISCHES BRIEFING · 2026

Mesh.

Strukturelle Quellcode-Kompression für LLM-Kontexteffizienz.

Edgar Baumann
Co-Founder · WU Wien

Philipp Horn
Co-Founder · WHU

74%	**3,9×**	**$3,90**
Ø Token-Reduktion	Kontextgewinn im 128k-Fenster	vs $15,00/MTok · Claude Opus 4.6

edgar.baumann@try-mesh.com · philipp.horn@try-mesh.com

KONTEXTDEGRADIERUNG

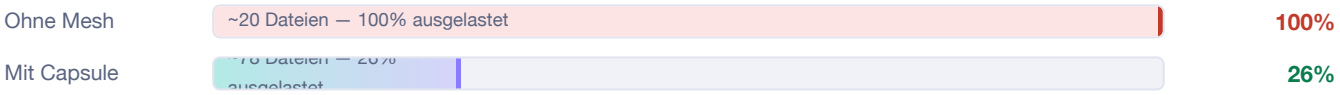
Mehr Kontext bedeutet nicht immer bessere Antworten — manchmal das Gegenteil.

LLMs arbeiten mit einem fixen Kontextfenster. Mit steigender Auslastung verschlechtert sich die Abrufgenauigkeit — das ist kein theoretisches Problem, sondern in mehreren unabhängigen Benchmarks gemessen.

NIAH — Needle In A Haystack

Standard-Retrieval-Benchmark: Eine bestimmte Information wird an verschiedenen Positionen in einem langen Dokument platziert; das Modell muss sie finden. Forschungsergebnisse (Liu et al., 2023, *„Lost in the Middle"*) zeigen:

- Modelle arbeiten zuverlässig bis ~60–70 % Kontextauslastung
- Darüber hinaus fällt die Abrufgenauigkeit um **20–40 %**, je nach Position der gesuchten Information
- Der Effekt verstärkt sich bei Kontexten über 100k Tokens



SWE-bench

Branchen-Benchmark für KI-Coding-Agenten, die echte GitHub-Issues lösen. Spitzenmodelle (Claude Opus 4.6, GPT-4o, Gemini 3.1 Pro) lösen rund **40–55 %** der Aufgaben. Die Root-Cause-Analyse von Fehlern zeigt immer wieder dasselbe Muster: **Der Agent hatte nicht genug Codebase-Kontext, um das Problem vollständig zu verstehen.**

Der Standardansatz — Embedding-basiertes Retrieval — wählt relevante Dateien aus und schickt sie dann roh ans Modell. Das löst das Problem nicht: Die Auswahl bestimmt, was geladen wird, aber was geladen wird, verbraucht weiterhin das volle Token-Budget. Je größer die Codebasis, desto weniger passt pro Fenster.

PERFORMANCE

Produktionspipeline · 5 Dateitypen · 6 Größenkategorien

TypeScript, YAML, SQL, HTML, Markdown — echte Zahlen aus der laufenden Capsule-Pipeline.

DATEIGRÖSSE	TOKEN-REDUKTION	EINSPARUNG
~200B (xs) — wird übersprungen		+10%
~1 KB (klein)		–83%
~5 KB (mittel)		–95%
~18 KB (groß)		–98,5%
~50 KB (XL)		–99,4%
~100 KB (XXL)		–99,9%
Durchschnitt (klein–XXL)		**–74% → 3,9×**

Kontextfenster-Wirkung (128k)

Ohne Mesh	~20 Dateien — 100% ausgelastet	100%
Mit Capsule	~78 Dateien — 26% ausgelastet	26%

Kostenanalyse (Claude Opus 4.6 — $15,00/MTok Input)

SZENARIO	GESENDETE TOKENS	API-KOSTEN	ABGEDECKTE DATEIEN
Roher Code	1.000.000	$15,00	~100 mittlere Dateien
Mit Capsule	**260.000**	**$3,90**	**Dieselben 100 Dateien**

Architekturvergleich

EIGENSCHAFT	EMBEDDING-RETRIEVAL (BRANCHENSTANDARD)	CAPSULE-KOMPRESSION (MESH)
Was das Modell sieht	Roher Dateiinhalt zum vollen Token-Preis	**Komprimierter struktureller Deskriptor**
Token-Kosten	Skalieren mit Dateianzahl × Dateigröße	**Konstant niedrig, unabhängig von Dateigröße**

EIGENSCHAFT	EMBEDDING-RETRIEVAL (BRANCHENSTANDARD)	CAPSULE-KOMPRESSION (MESH)
Kontextfenster-Abdeckung	Begrenzt auf rohen Fit	**3,9× mehr Dateien pro Fenster**
Codebase-Größen-Sensitivität	Hoch — Abdeckung sinkt mit Wachstum	**Niedrig — Kompressionsrate steigt mit Dateigröße**
Gegenseitig ausschließend?	Nein — Capsule ist additiv zu jeder Retrieval-Strategie	

edgar.baumann@try-mesh.com · philipp.horn@try-mesh.com · Technische Demo auf Anfrage